Joins

- → Inner Joins
- → Left Join
- → RIGHT JOIN
- → OUTER JOIN
- → SELF JOIN

T1

id		

T2

Left Table

L1	L2	Key

+

Right Table

Key	R1	R2

INNER JOIN

Resultant Table

Key	L1	L2	R1	R2

Put a condition → where columns match → they append to resultant table

Tables
- customer
- Account

using dot

SYNTAX: →

Table

column name

```
SELECT customer.id, customer.first_name, customer.last_name,
       Account.id , Account.balance
  FROM customer
  INNER JOIN Account
  ON customer.id = Account.customer.id
```

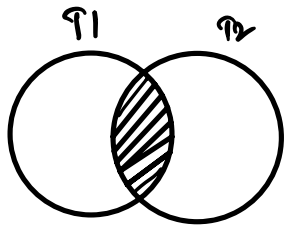
To simplify it (using AS)

```
select c.id, c.first_name, c.last_name a.balance
FROM customer AS c INNER JOIN ACCOUNT AS a
ON c.id = a.customer_id
```

Normal JOIN is equivalent to INNER JOIN.

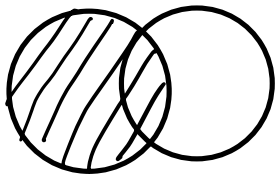
⇒ SELECT * FROM T_A JOIN T_B USING (Col_name);

Equals " " " " INNER JOIN " " " ;



LEFT JOIN

all rows available on left are also on resultant



```
SELECT a.*, t.*
FROM Account a
LEFT JOIN
        Transaction t
ON a.id = t.account_id;
```

- List out all the Accounts having transactions done b/w 2020-01-01 to 2021-01-01;

```
SELECT a.*, t.*
FROM Account a
LEFT JOIN Transaction t
ON a.id = t.account_id
WHERE t.date BETWEEN '2020-..' & '...'
```

RIGHT JOIN

- Opposite of left Join

```
SELECT c.*, b.address, b.id
FROM Customer c
RIGHT JOIN
    Branch b
ON
    b.id = c.branch_id;
```

The order of joins is important because it determines how the tables are combined and which records are included in the result. In this problem:

- Starting with the Buyer table ensures that all buyers are considered, even if they haven't made a purchase.
- RIGHT JOIN with the Seller table ensures that all sellers are included, even if they haven't sold anything.
- RIGHT JOIN with the Product table ensures that all products are included, even if they haven't been purchased.

This order ensures that the query captures all possible combinations of buyers, sellers, and products, fulfilling the requirement to check if any sellers have sold products.

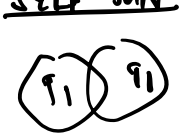
[There is usually no much use of cross JOIN)

CROSS JOIN (Cartesian Product)

```
SELECT i.*, c.id FROM INSURANCE i
        CROSS JOIN customer c;
```

Not meaning ful.

SELF JOIN



```
SELECT a1.id,
       a2.balance
FROM
       Account as a1
INNER JOIN
       Account as a2
ON a1.id = a2.id;
```

FULL OUTER JOIN

Emulate (left JOIN UNION RIGHT JOIN)

```
SELECT
    a.*, t.*
FROM
    account a
LEFT JOIN
    transaction T
ON  t.account_id = a.id;
```

Copy → Paste → UNION

Replace by Right JOIN

⇓

we'll get a/b values.